

ARCHITECTURE ATLAS

Cross-Feature Fusion Mamba Networks, Drawn Block by Block

*From YOLOv1 and Mamba to the visible–thermal
detector built in this project,
and the system it is meant to become*

Ritesh Roshan

Version 1.0
October 2026

Contents

1	How to read this atlas	1
1.1	One visual language	1
1.2	Arrows, shapes and symbols	2
1.3	What is redrawn and what is ours	2
2	YOLO, from one grid to YOLO26	3
2.1	The original idea	3
2.2	What changed between versions	3
2.3	YOLO26-n, the detector this project builds on	6
3	State-space models and Mamba	9
3.1	The state-space model	9
3.2	The Mamba block and its two-dimensional form	11
3.3	What CFFM-Net takes from these models	13
4	CFFM-Net	14
4.1	The whole network	14
4.2	Inside the fusion block	16
4.3	The gated recurrence	17
4.4	The numbers	18
4.5	Versions	18
5	Training and evaluation	19
5.1	From raw data to results	19
5.2	The notebooks	21
6	Beyond still images	23
6.1	Temporal State Memory	23
6.2	The whole project	25
A	Figure sources	27

How to read this atlas

This atlas collects every architecture and workflow diagram of the project in one place. It begins with the models CFFM-Net is built from, redrawn from their original papers: the YOLO family from its first version to YOLO26, and the state-space models that lead to Mamba. It then draws CFFM-Net itself, the way it is trained and evaluated, and the system it is meant to grow into. Every number in the diagrams of CFFM-Net was read from the trained model or from its configuration, not estimated.

1.1 One visual language

All diagrams share the conventions of the Transformer paper [8]: rounded blocks with a light fill and a thin outline, grey containers marked $N\times$ for blocks that repeat, small circles for element-wise operations, and thin arrows for data. Diagrams of whole networks follow the layout of recent multispectral detection papers such as CFMW [7]: dashed regions group the parts of the network, real images and feature maps sit where the data flows, and tensor shapes are written beside the layers that produce them.

Colour always means the same thing.

colour	meaning
blue	the visible camera, its backbone and its feature maps
orange	the thermal camera and its backbone
green	fusion by weighted sum, the neck, and outputs that are kept
purple	selective scans and state: the Mamba block, CMFM and the temporal memory
gold	detection heads, attention blocks and evaluation
pink	linear projections and embeddings
brown	data and the steps that prepare it
grey	convolutions and plain bookkeeping such as concatenation and upsampling

1.2 Arrows, shapes and symbols

A solid arrow carries data. A dashed arrow carries something that exists only in training, is optional, or is planned. The circles mark element-wise operations: $+$ adds, $\times$ multiplies, $\parallel$ concatenates along the channel axis and $\div$ splits it. A shape written as 128×80^2 means 128 channels on an 80 by 80 grid, for a 640-pixel input; the stride of a level is the input size divided by its grid size.

1.3 What is redrawn and what is ours

Chapters 2 and 3 redraw published architectures in the common style above. They follow the original papers in structure and naming, and they use this project's images only to make the data flow concrete. Chapters 4 to 6 are the project's own: CFFM-Net, its training and evaluation, and its roadmap. Appendix A lists the source of every figure.

YOLO, from one grid to YOLO26

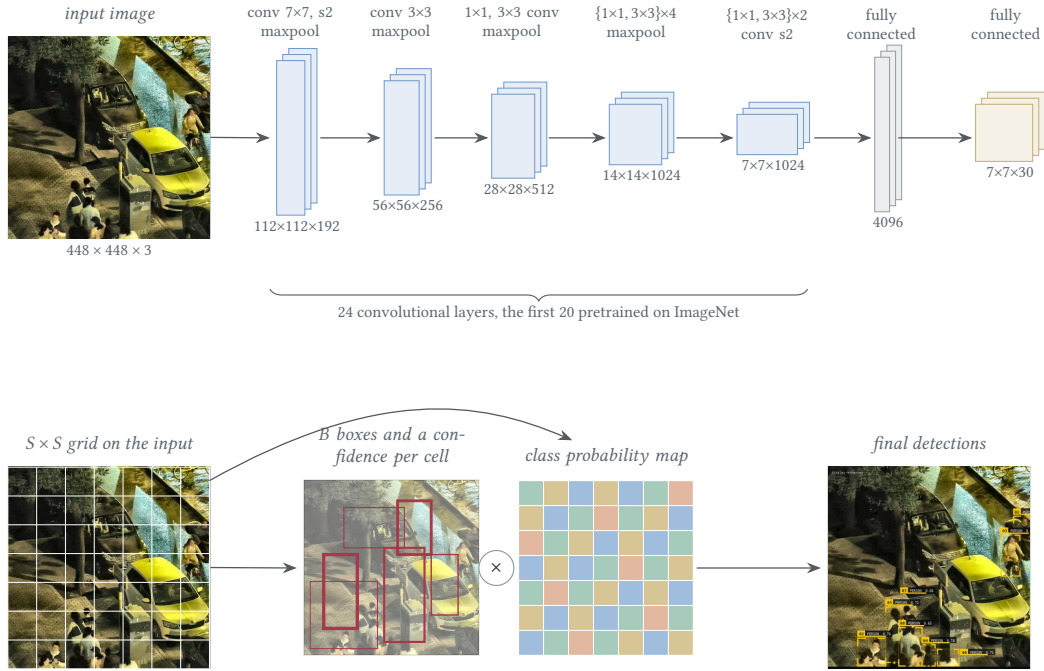
2.1 The original idea

YOLO [1] turned detection into a single regression. The image is divided into an $S \times S$ grid. Each cell predicts a few boxes, each with a confidence, and one set of class probabilities. A convolutional network reads the whole image once and writes all of these numbers into one output tensor (Figure 2.1). Because there is no separate region-proposal stage, the detector runs in real time, which is why every later version kept the name.

The weaknesses of this first design shaped everything that followed. One coarse grid means a cell can hold only one object, so small objects that share a cell are lost. The fully connected head fixes the input size. And duplicate boxes still have to be removed after the network by non-maximum suppression (NMS), a step that is neither learned nor parallel.

2.2 What changed between versions

Figure 2.2 summarises the steps that matter for this project. YOLOv3 [9] predicted at three scales with a feature pyramid [10], which is the first answer to small objects. YOLOv4 [11] and YOLOv5 [12] added cross-stage partial backbones, spatial pyramid pooling and the path-aggregation neck [13]. YOLOv8 [14] dropped anchor boxes, split the head into box and class branches, assigned targets by task alignment [15], and regressed boxes as distributions (DFL) [16]. YOLOv10 [17] removed NMS by training two heads, one that assigns many predictions to each object and one that assigns exactly one, and keeping only the second at test time.



Each of the $S \times S$ cells predicts B boxes, each with a confidence, and one set of C class probabilities. Their product, thresholded and de-duplicated with non-maximum suppression, gives the detections. The output tensor is $S \times S \times (5B + C)$: $7 \times 7 \times 30$ for $S = 7$, $B = 2$, $C = 20$.

FIGURE 2.1. YOLOv1, redrawn after Redmon et al. [1]. Top: the network, 24 convolutional layers followed by two fully connected ones, ending in a $7 \times 7 \times 30$ tensor. Bottom: how that tensor is read. Each grid cell proposes boxes with confidences and a class distribution; their product, after non-maximum suppression, gives the final detections.

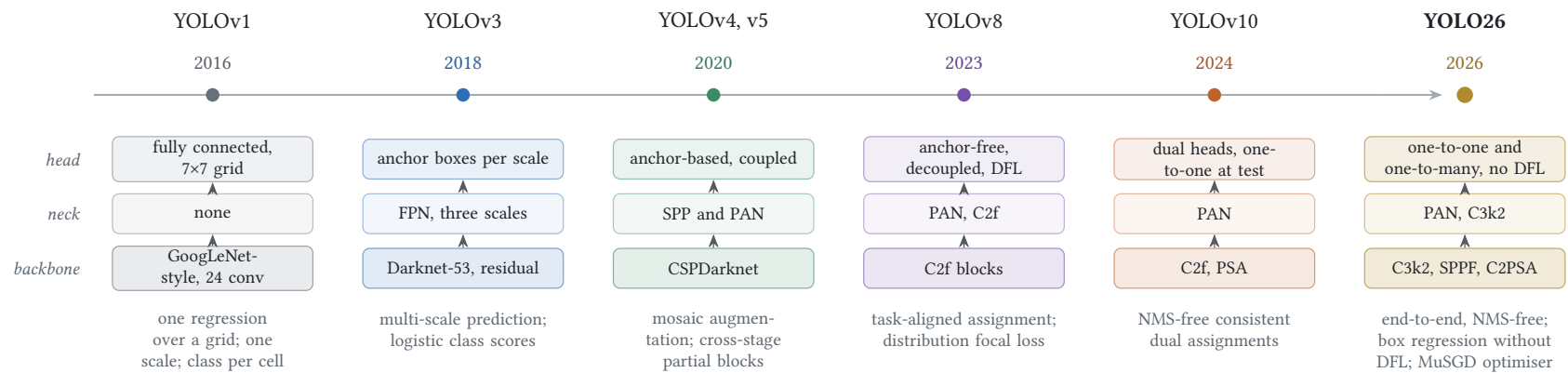


FIGURE 2.2. Six generations of YOLO, each reduced to backbone, neck and head, with the change that defines it. The design converges on a multi-scale pyramid, a path-aggregation neck and an end-to-end head without NMS.

2.3 YOLO26-n, the detector this project builds on

YOLO26 [3] keeps YOLOv10's end-to-end head and removes the distribution focal loss, regressing box edges directly. Figure 2.3 draws its smallest scale, YOLO26-n, exactly as configured in Ultralytics 8.4.171, with the tensor shapes measured on a 640-pixel input. It has 2.57 million parameters and costs 6.1 GFLOPs.

The backbone halves the resolution five times. Its outputs at strides 8, 16 and 32, called P3, P4 and P5, feed the neck. The neck first passes information downwards, from the coarse, semantic P5 to the fine P3, by upsampling and concatenation, then upwards again by strided convolution, so that every output level has both fine detail and context. Three detection heads read the levels at strides 8, 16 and 32. In training, each head is supervised twice: a one-to-many branch assigns up to ten predictions to each object, which gives dense gradients, and a one-to-one branch assigns exactly one, which teaches the network not to produce duplicates. At test time only the one-to-one branch runs, and the top 300 boxes are the answer, with no NMS.

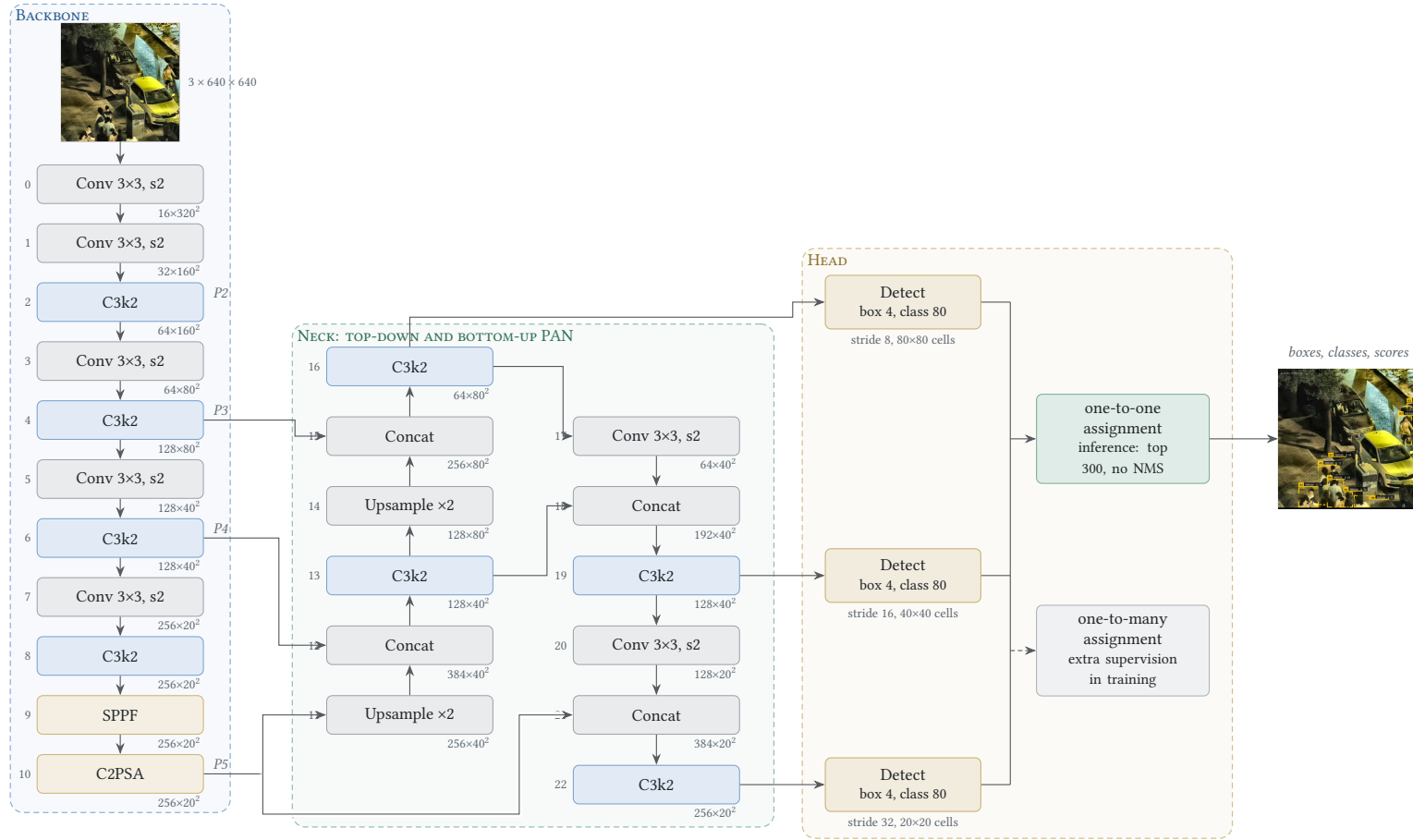


FIGURE 2.3. YOLO26-n as used in this project. Blue: backbone; green: the two-way PAN neck; gold: the end-to-end heads. Numbers left of each block are its layer index; shapes are channels × height². The one-to-many branch is used only in training.

The four repeated blocks are drawn in Figure 2.4. C3k2 is a cross-stage partial block: it splits the channels, sends one half through a stack of bottlenecks and concatenates the halves again, so gradients have a short path. SPPF pools the same map three times with a 5×5 window to mix context from growing regions. C2PSA adds self-attention [8] at the coarsest level, where the map is small enough for attention to be cheap.

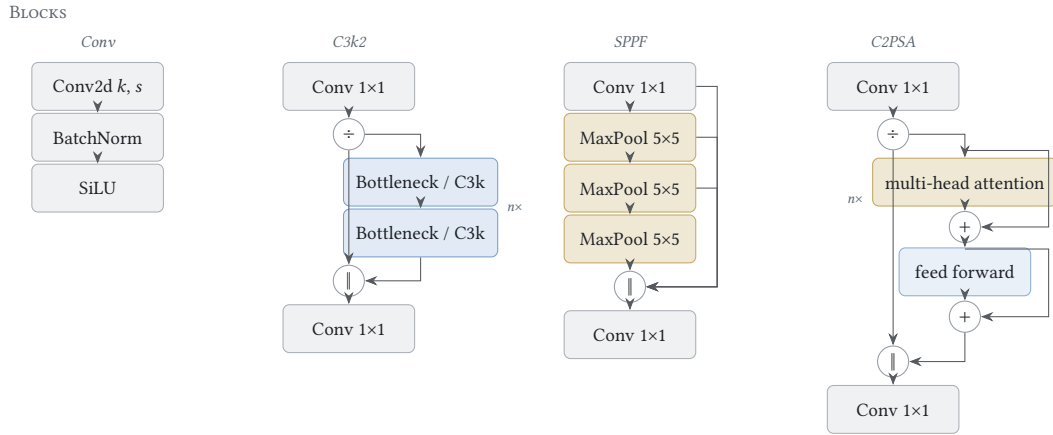


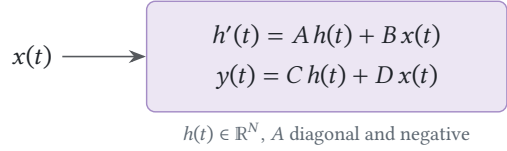
FIGURE 2.4. The building blocks of YOLO26: the standard convolution, the cross-stage partial C3k2, fast spatial pyramid pooling (SPPF), and C2PSA with multi-head attention and a feed-forward layer.

State-space models and Mamba

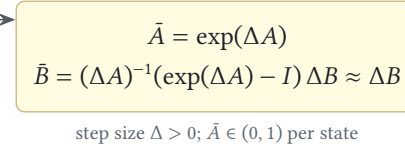
3.1 The state-space model

A state-space model reads a sequence and keeps a small hidden state h that summarises everything seen so far (Figure 3.1). In continuous time the state evolves as $h'(t) = A h(t) + B x(t)$ and the output is read from it as $y(t) = C h(t) + D x(t)$. To apply this to tokens, the equation is discretised with a step size Δ : the state then decays by $\tilde{A} = \exp(\Delta A)$ and takes in $\tilde{B}x$ at every step. S4 [4] showed that with a structured A such models capture very long sequences, and that the same computation can be run either as a recurrence, one token at a time, or as one long convolution.

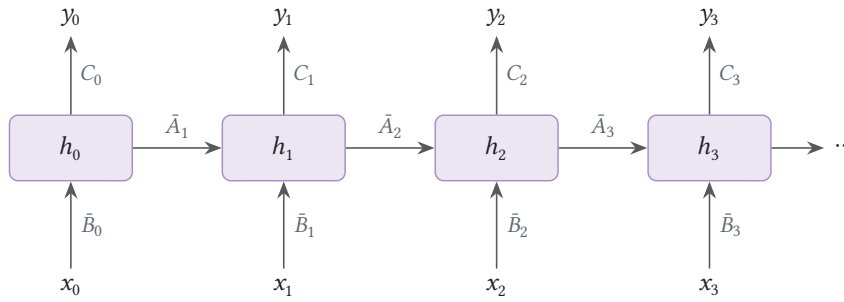
(A) CONTINUOUS STATE SPACE



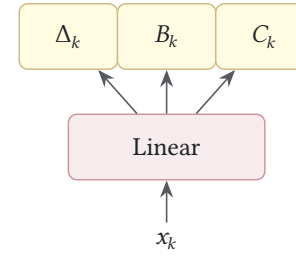
(B) DISCRETISATION, ZERO-ORDER HOLD



(C) RECURRENT VIEW, ONE STEP PER TOKEN



(D) SELECTION (MAMBA)



$\Delta_k \rightarrow 0$: $\bar{A}_k \rightarrow 1$, $\bar{B}_k \rightarrow 0$, the state is held and x_k ignored.

Δ_k large: $\bar{A}_k \rightarrow 0$, the past is forgotten and the state rewritten from x_k .

Because Δ, B, C depend on the input, the model chooses per token what to keep. This is the knob CFFM-Net scales by sensor reliability.

$h_k = \bar{A}_k h_{k-1} + \bar{B}_k x_k$, $y_k = C_k h_k + D x_k$. One pass over L tokens costs $O(L)$, and the state h is all the memory the model keeps.

FIGURE 3.1. From a continuous state-space model to a selective scan, after Gu et al. [4] and Gu and Dao [5]. (a) The continuous system. (b) Discretisation with step Δ . (c) The recurrent view: one state update per token, at linear cost. (d) Selection: Δ , B and C are computed from each token, so the model decides what to keep.

Mamba [5] made the step size and the projections depend on the input. This one change is the whole idea, and panel (d) of Figure 3.1 spells out why it matters. When a token produces a small Δ , the decay $\bar{A}$ is close to one and the input term close to zero, so the state is held and the token is ignored. When it produces a large Δ , the past decays away and the state is rewritten from the token. The model can therefore learn, token by token, what to remember and what to skip.

3.2 The Mamba block and its two-dimensional form

Figure 3.2 draws the Mamba block beside its adaptation to images. The block expands the channels, mixes neighbouring tokens with a short convolution, runs the selective scan, and multiplies the result by a gate computed from the input, a structure close to the gated feed-forward layers of Transformers. Blocks are stacked with a normalisation in front and a residual connection around each.

An image is not a sequence, and flattening it row by row breaks its neighbourhoods. VMamba [6] answers this with a two-dimensional scan, SS2D: the same feature map is read in four orders, along rows and columns in both directions, each order is scanned separately, and the four results are added back on the grid. Every pixel then receives context from all four directions at linear cost. Vision Mamba [18] follows a related bidirectional design, and MambaVision [19] shows that pairing the scan with a plain convolutional branch helps it to keep local detail.

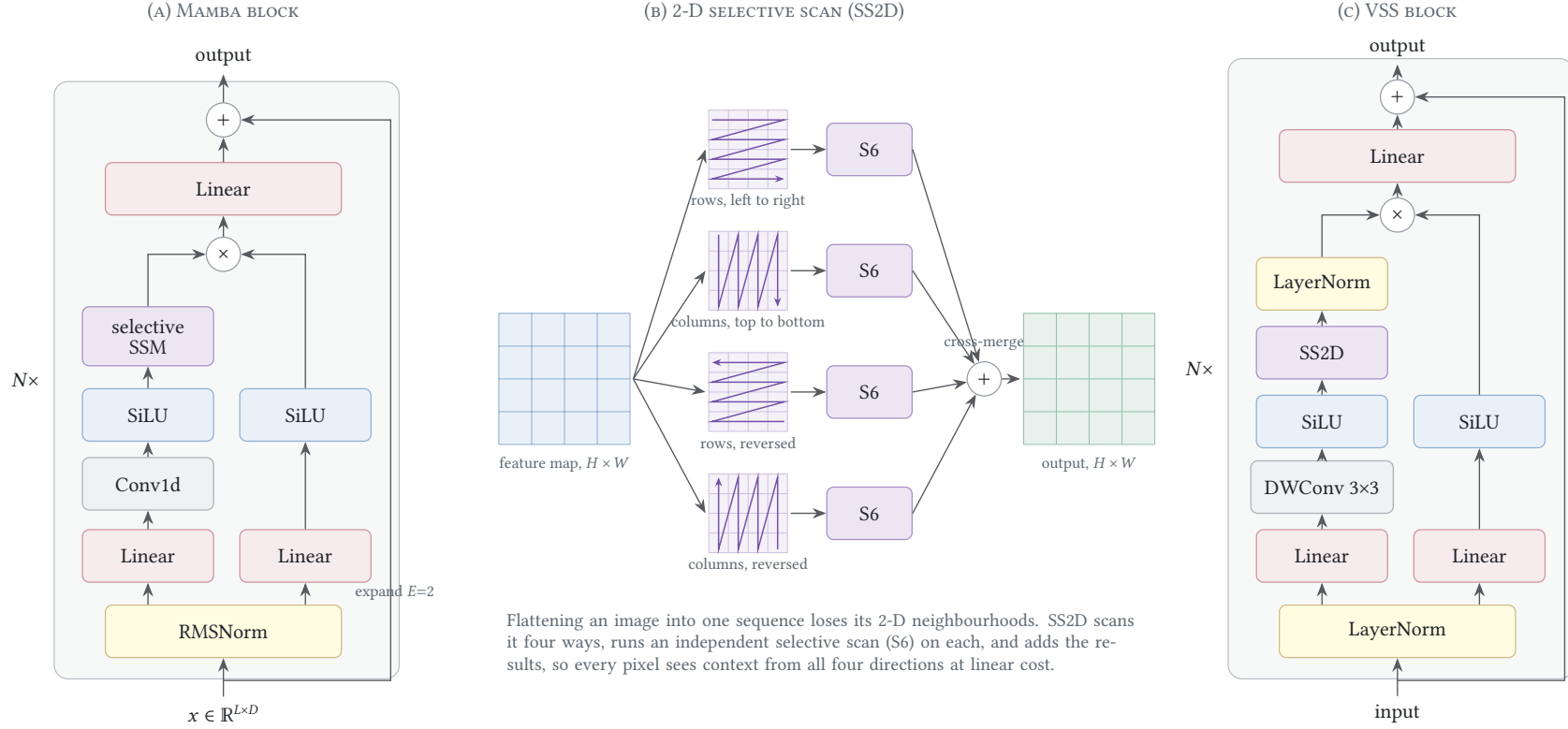


FIGURE 3.2. (a) The Mamba block, after Gu and Dao [5]. (b) The four-way cross-scan of VMamba [6], with an independent selective scan (S6) per direction and a cross-merge. (c) The VSS block that wraps SS2D, after Liu et al. [6].

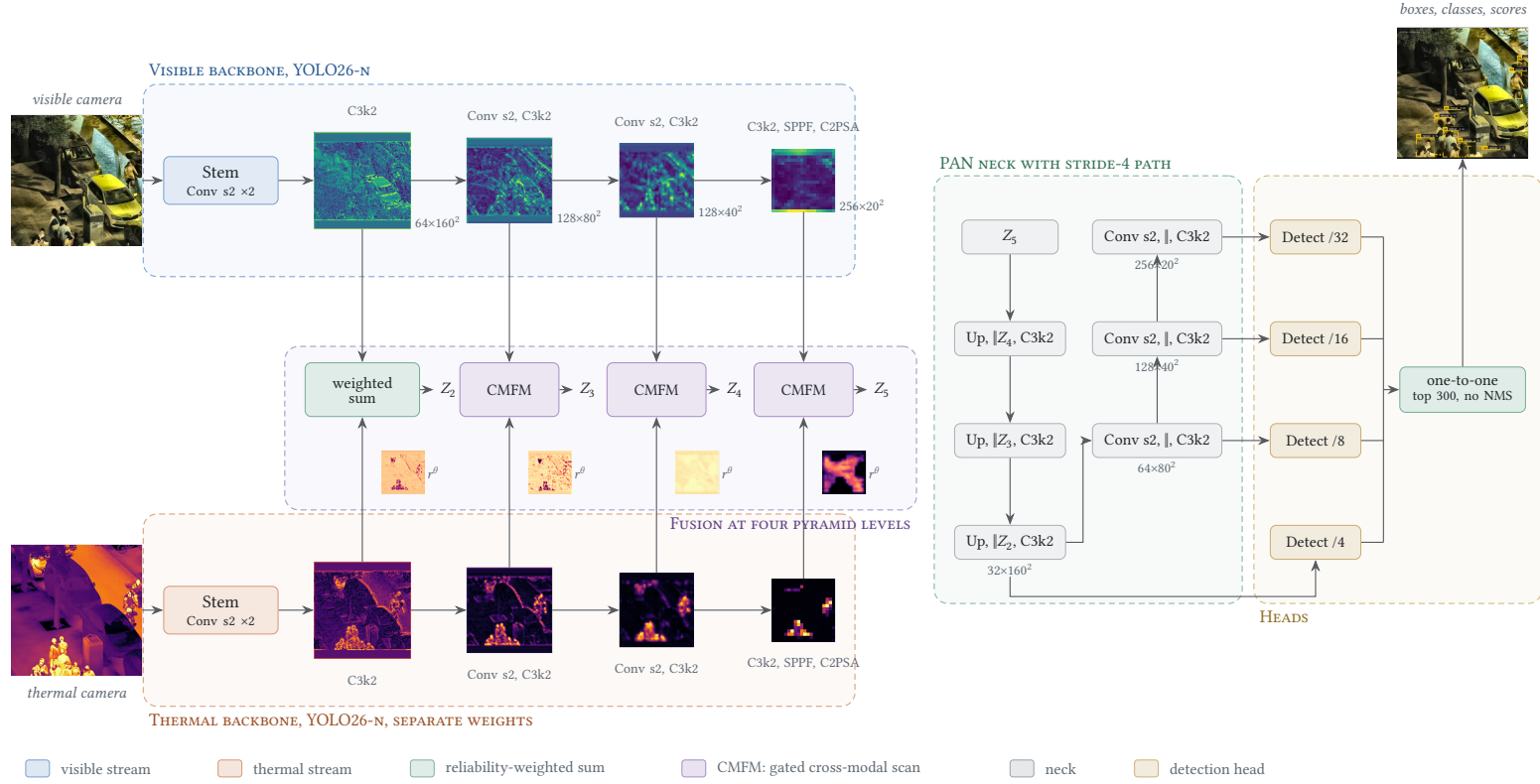
3.3 What CFFM-Net takes from these models

CFFM-Net borrows four specific pieces and adds one of its own. From Mamba it takes the input-dependent step size. From VMamba it takes the four-direction scan over a two-dimensional map. From MambaVision it takes a local convolutional branch beside the scan. From the practice of zero-initialised residual branches it takes an output projection that starts at zero, so a new block begins as the identity of its residual path. The addition is the reliability gate: CFFM-Net multiplies each token’s step size by how reliable its camera is at that location, which lets the scan skip the tokens of a camera that cannot be trusted. Chapter 4 draws how.

CFFM-Net

4.1 The whole network

CFFM-Net detects objects from a visible and a thermal camera at once (Figure 4.1). Each camera has its own YOLO26-n backbone with separate weights; both start from the same COCO-pretrained weights, the thermal image repeated to three channels. The two backbones run side by side, and at four pyramid levels, P2 to P5, their feature maps are fused. At P2, stride 4, the fusion is a reliability-weighted average, because a scan over a 160×160 map buys little for its cost. At P3, P4 and P5 it is the cross-modal fusion Mamba block, CMFM. The fused maps Z_2 to Z_5 then enter an unchanged YOLO26 neck extended with a stride-4 path, and four heads predict boxes at strides 4, 8, 16 and 32.



Feature maps are channel-averaged activations of the trained model on this LLVIP test pair; r^θ is its thermal reliability at each fusion level. Shapes are channels $\times$ height² for a 640-pixel input. 6.03 M parameters: 1.37 M per backbone, 2.15 M in the fusers.

FIGURE 4.1. CFM-Net as trained in the pilot, drawn with the real data of one LLVIP test pair. The feature maps are the channel-averaged activations of each backbone stage; the small maps r^θ are the thermal reliability the model assigned at each fusion level. 6.03 million parameters: 1.37 million in each backbone, 2.15 million in the fusion blocks and the rest in the neck and heads.

The fusion never feeds back into the backbones. Each stream keeps its own features all the way to P5, so one camera’s failure cannot corrupt the other’s representation, and the fusion blocks decide how much of each to pass to the neck. Figure 4.2 gives the same network in compact form.

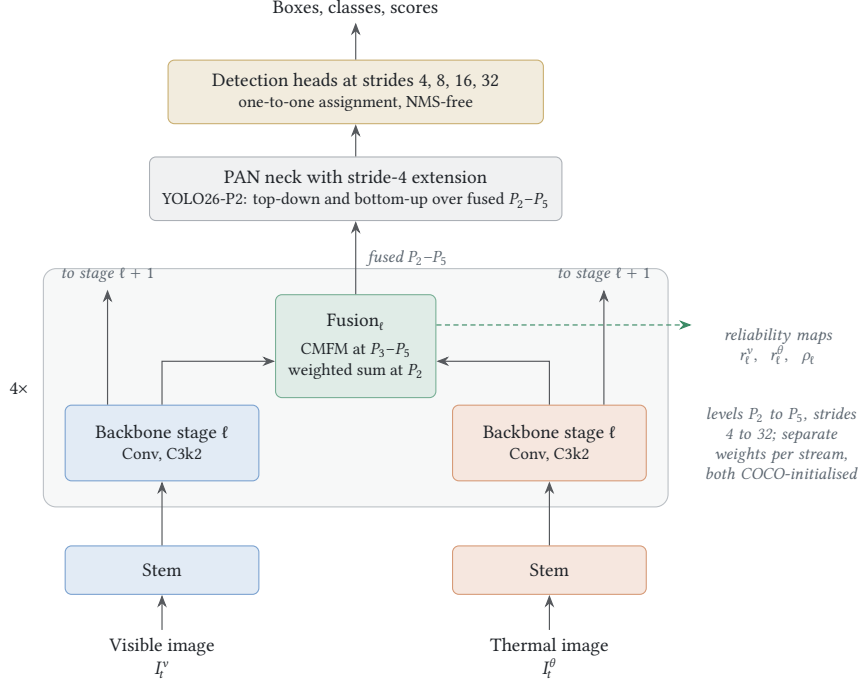


FIGURE 4.2. CFFM-Net in compact form: one backbone stage per stream, the fusion at each level, the shared neck and the heads. The block repeats four times, once per pyramid level.

4.2 Inside the fusion block

Figure 4.3 opens CMFM. A small reliability head looks at both feature maps and at their difference and outputs two maps, r^v and r^θ , between zero and one: how far the block should trust each camera at each location. The thermal features are first shifted by a learned offset field of at most four cells, which corrects small misregistration between the cameras. Both streams are projected to half their channels, then interleaved token by token, $x_1^v, x_1^\theta, x_2^v, x_2^\theta, \dots$, so that the scan sees both cameras at the same place one after the other. The gated scan runs over this sequence in four directions, its output is merged and normalised, joined with a local depthwise branch, and projected back. Because that last projection starts at zero, a new block computes exactly the reliability-weighted average of its inputs, and training can only add to it.

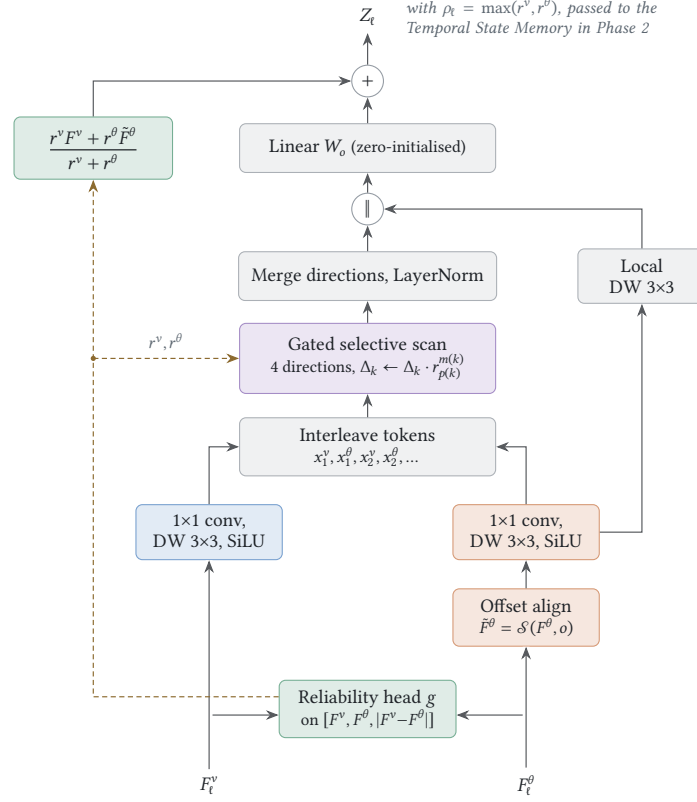


FIGURE 4.3. The cross-modal fusion Mamba block. The reliability maps are used twice: they gate the scan and they weight the residual average. The output projection W_o starts at zero.

4.3 The gated recurrence

The single change that makes the scan selective between cameras is drawn in Figure 4.4. Each token’s step size Δ_k is multiplied by the reliability r_k of the camera that produced it. When r_k approaches zero, Δ_k does too, so the decay approaches one and the input term zero: the token neither writes into the state nor erases it. A dazzled or missing camera is skipped, and the memory built from the other camera carries on. The unit tests check this property directly: with thermal reliability set to zero, changing the thermal features does not change the visible outputs.

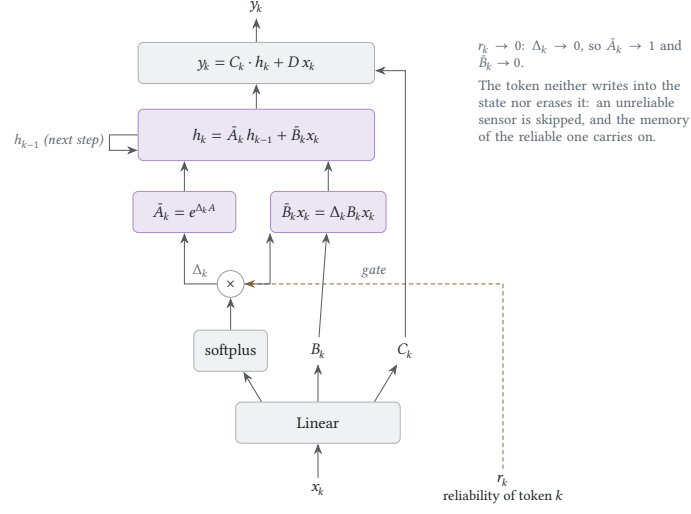


FIGURE 4.4. One step of the reliability-gated selective scan. The reliability r_k scales the step size, which is the knob Mamba uses to decide what to keep.

4.4 The numbers

model	parameters	GFLOPs	latency, T4 FP16
YOLO26-n, one camera	2.50 M	5.9	17.0 ms
two-stream concat	4.07 M	9.7	23.4 ms
CFFM-Net, pilot	6.03 M	18.4	41.9 ms

Latency is measured at batch 1 on one T4 with the PyTorch model, before any engine optimisation. CFFM-Net costs 1.79 times the concat baseline, inside the limit of two set before training.

4.5 Versions

The pilot version uses YOLO26’s stride-4 head. A head-control experiment showed that this head, not the fusion, cost about 1.8 AP on LLVIP, so the second version, CFFM-Net v2, returns to the standard three-level head. It also trains with modality dropout, blacking out one camera at random so the model learns to work without it, and keeps the fusion weights in single precision with a floor on their sum, which removed a numerical overflow. In the first clean comparison on M3FD, CFFM-Net v2 reached 0.488 AP against 0.483 for the concat baseline trained the same way.

Training and evaluation

5.1 From raw data to results

Figure 5.1 follows the data through the pipeline. The LLVIP [2] and M3FD [20] archives are converted once into one paired layout: two folders of images with identical names, the long side at 640 pixels, and labels in YOLO format. A paired loader reads both images and stacks them into one four-channel tensor, visible colour plus thermal grey. Augmentation is shared, so a crop or a flip moves both cameras together, except for colour jitter, which touches only the visible image. Modality dropout then blacks out one camera with probability 0.15 each, and in half of those cases tells the model which camera is gone, so that both the silent and the reported failure are seen in training.

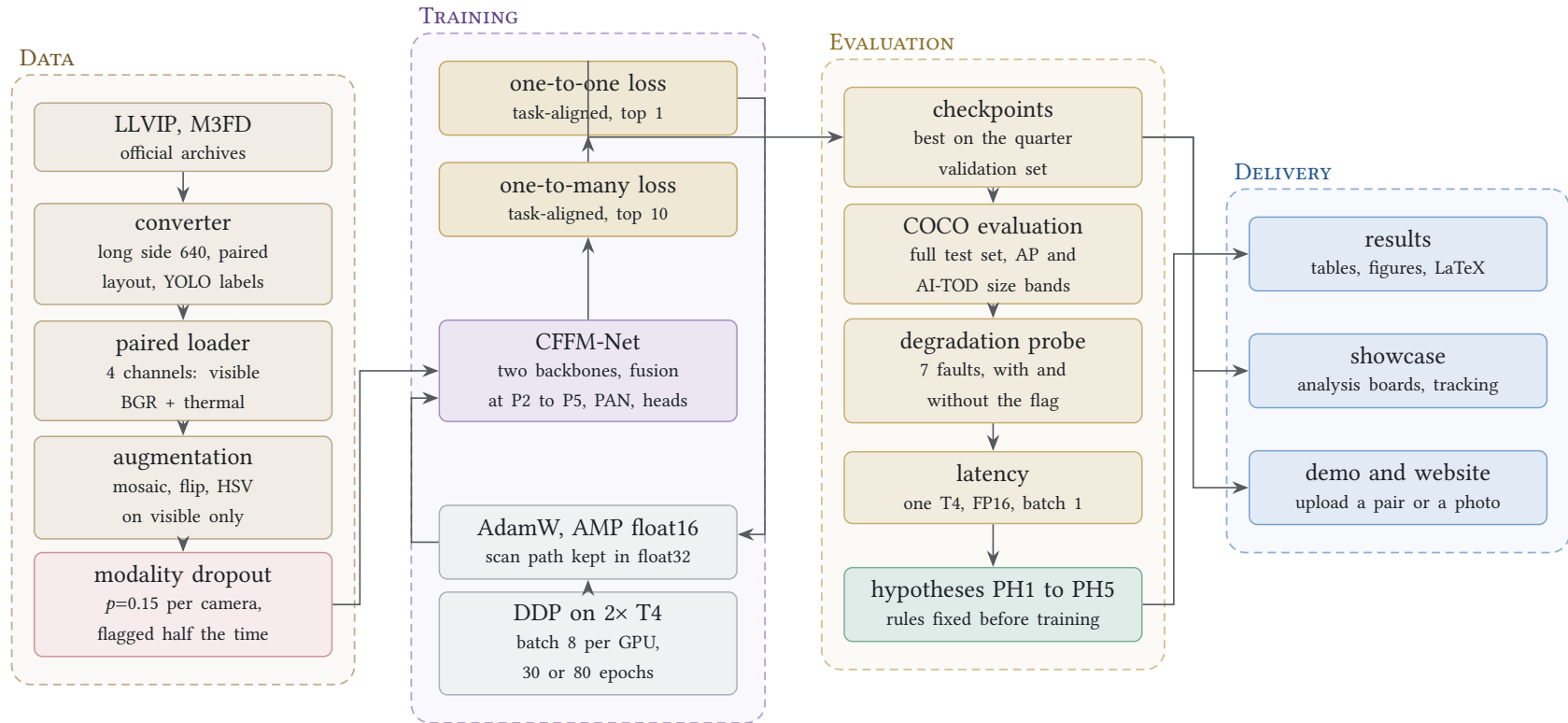


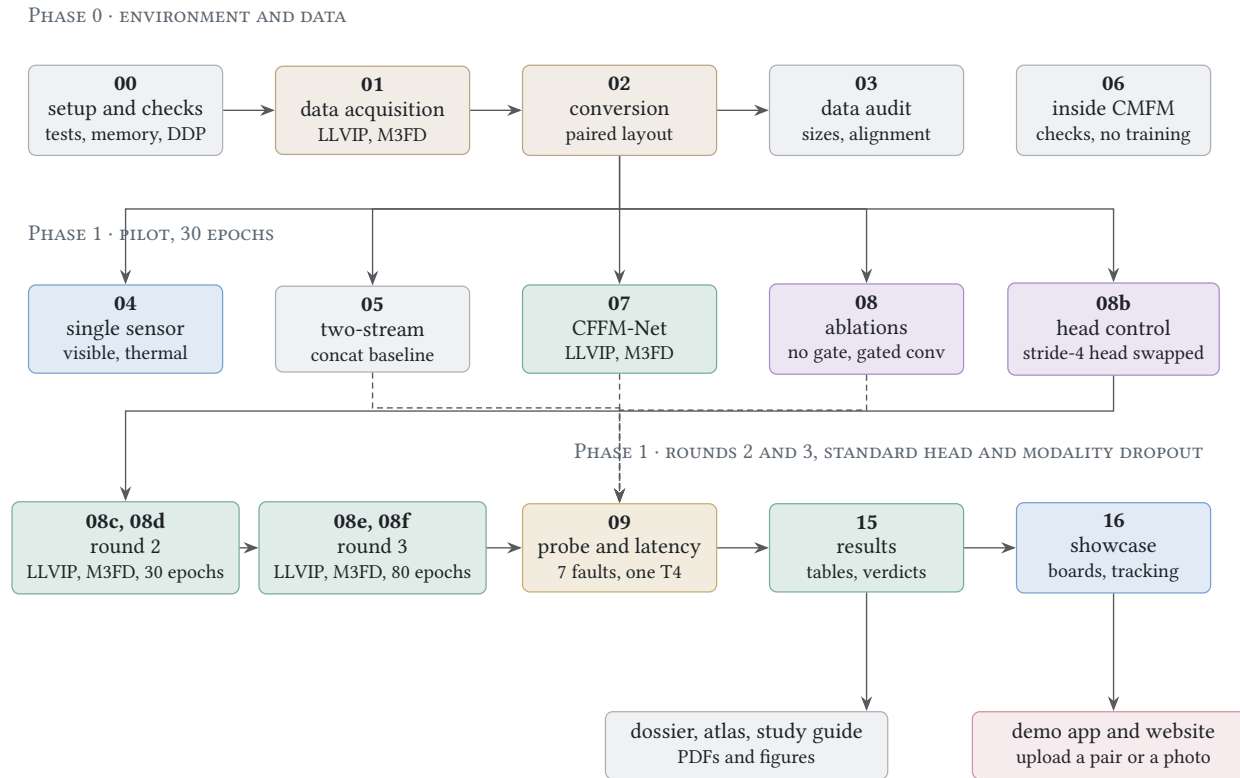
FIGURE 5.1. The training and evaluation pipeline. Brown: data; purple: training; gold: evaluation; blue: what the project delivers. The loop between the losses and the optimiser is one training step.

Training follows YOLO26: two task-aligned losses, one-to-many with ten predictions per object and one-to-one with a single one, summed. The optimiser is chosen by Ultralytics and was AdamW at this schedule. Mixed precision runs the network in 16-bit floats, but the whole scan path, from the projections to the fusion weights, stays in 32-bit floats, because a recurrence over twelve thousand tokens overflows the 16-bit range. Two T4 GPUs share each batch through distributed data parallelism.

Every trained checkpoint goes through the same evaluation. COCO average precision is computed on the full test set, with the AI-TOD size bands that separate very tiny, tiny and small objects. A degradation probe applies seven sensor faults at test time, from a darkened visible image to a thermal camera shifted by sixteen pixels. Latency is measured on one T4. The five hypotheses of the pilot are then decided by rules written down before training.

5.2 The notebooks

The work is organised as notebooks, each one step of Figure 5.2. Phase 0 sets up and checks the environment and prepares the data. Phase 1 trains the single-camera and concat baselines, CFFM-Net, its ablations and the head control in the pilot, then the improved version in two further rounds, and closes with the probe, the results and a showcase of the trained models.



Hypotheses decided in 15: PH1 fusion beats thermal alone · PH2 gating adds value · PH3 small objects on M3FD · PH4 the scan is needed · PH5 latency at most twice the baseline. Solid arrows carry data and checkpoints; dashed arrows carry checkpoints read by the probe.

FIGURE 5.2. The notebooks and how their outputs flow. Each training notebook runs as an independent Kaggle session; the probe reads their checkpoints, and the results notebook decides the hypotheses.

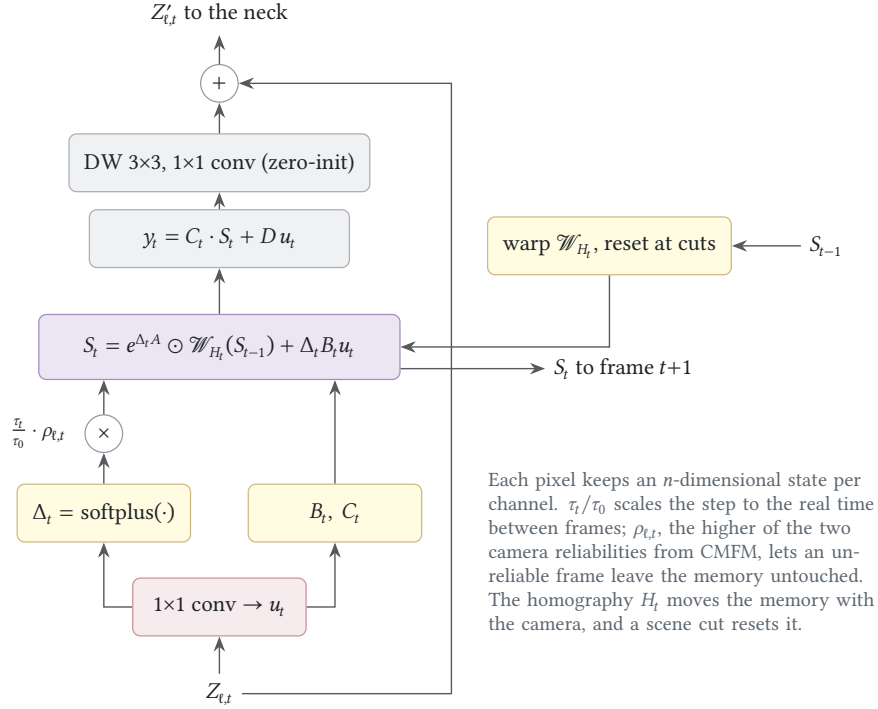
Beyond still images

6.1 Temporal State Memory

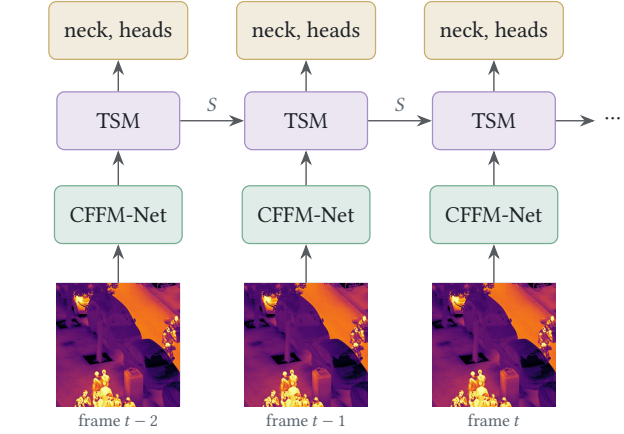
Video adds a dimension the still-image model ignores: a person who is hard to see in one frame was usually visible in the previous one. Phase 2 adds a Temporal State Memory (TSM) between each fusion block and the neck (Figure 6.1). Every pixel of every pyramid level keeps a small state, updated once per frame by the same selective recurrence as the scan, so the network can carry evidence forward in time.

Three signals control the update. The ratio τ_t/τ_0 scales the step to the real time between frames, so a dropped frame counts as a longer step instead of being ignored. The reliability $\rho_{\ell,t}$, the higher of the two camera reliabilities from CMFM, lets a frame in which both cameras are unreliable leave the memory untouched. And the homography H_t between consecutive frames moves the memory with the camera, so the state stays attached to the scene, while a scene cut resets it. As in CMFM, the output projection starts at zero, so an untrained memory changes nothing. The module is implemented and tested; training it is the first task of Phase 2.

(A) ONE FRAME AT ONE PYRAMID LEVEL



(B) UNROLLED OVER A STREAM



The fused features of each frame update a per-pixel memory, so a person who drops below the threshold in one frame is still remembered in the next. Phase 2 trains this on paired visible-thermal video.

FIGURE 6.1. The Temporal State Memory of Phase 2. (a) One update at one pyramid level. (b) The same module unrolled over a stream: the state S passes from frame to frame.

6.2 The whole project

Figure 6.2 places everything in this atlas on the project’s roadmap. Phase 0 and the pilot of Phase 1 are done, and the long training runs of Phase 1 are under way. Phase 2 turns the detector into a video analyser with tracking, built on the memory above and on ByteTrack [21]. Phase 3 connects live cameras, where the difficulty is no longer accuracy but time: one capture thread per camera, only the newest frame kept, an optimised engine and a monitor that raises the sensor flags when a camera stalls. Phase 4, optional, combines several cameras on a shared ground plane and reports only anonymous counts, crossings and dwell times, never identities.

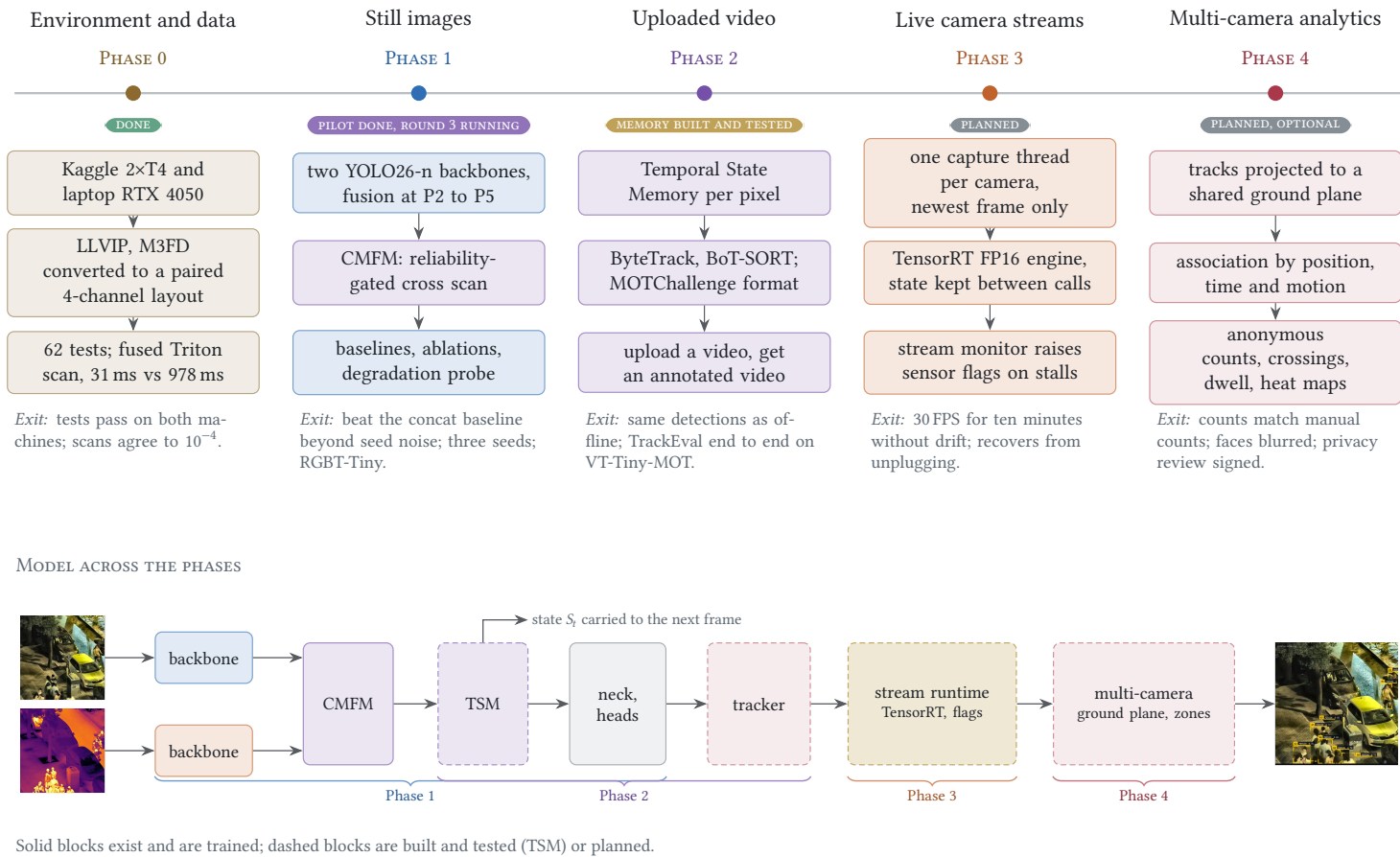


FIGURE 6.2. The full scope of the project. Top: the five phases with what each delivers and the test it must pass. Bottom: how the model grows across the phases; dashed blocks are built and tested or still planned.

Figure sources

figure	source
2.1	redrawn after Redmon et al. [1]; images from the LLVIP test set [2]
2.2	summarised from the papers and repositories cited in Section 2.2
2.3	drawn from the YOLO26 configuration in Ultralytics 8.4.171 [3]; shapes measured
2.4	drawn from the Ultralytics implementation of each block
3.1	redrawn after Gu et al. [4] and Gu and Dao [5]
3.2	redrawn after Gu and Dao [5] and Liu et al. [6]
4.1	this project; layout after CFMW [7]; feature and reliability maps from the trained model
4.2 to 4.4	this project, drawn from its source code
5.1, 5.2	this project
6.1, 6.2	this project; phases as defined in the research dossier

All figures are TikZ sources in `cffm-net-pilot/docs/figures` and are rebuilt with `build_figures.py`; the images they show are regenerated from the trained checkpoints.

Bibliography

- [1] J. Redmon, S. Divvala, R. Girshick and A. Farhadi. You only look once: Unified, real-time object detection. In *CVPR*, 2016. arXiv:1506.02640.
- [2] X. Jia, C. Zhu, M. Li, W. Tang, and W. Zhou. LLVIP: A visible-infrared paired dataset for low-light vision. In *Proc. ICCV Workshops*, 2021.
- [3] Ultralytics. YOLO26. Released January 2026; technical report *arXiv preprint* arXiv:2606.03748, 2026. <https://docs.ultralytics.com/models/yolo26>.
- [4] A. Gu, K. Goel, and C. Ré. Efficiently modeling long sequences with structured state spaces. In *Proc. ICLR*, 2022.
- [5] A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint* arXiv:2312.00752, 2023.
- [6] Y. Liu, Y. Tian, Y. Zhao, H. Yu, L. Xie, Y. Wang, Q. Ye, and Y. Liu. VMamba: Visual state space model. In *Advances in NeurIPS*, 2024. arXiv:2401.10166.
- [7] H. Li, Q. Hu, B. Zhou, Y. Yao, J. Lin, K. Yang, and P. Chen. CFMW: Cross-modality fusion Mamba for robust object detection under adverse weather. *IEEE Trans. Circuits and Systems for Video Technology*, 35(12):12066–12081, 2025. doi:10.1109/TCSVT.2025.3587918.
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017. arXiv:1706.03762.
- [9] J. Redmon and A. Farhadi. YOLOv3: An incremental improvement. *arXiv preprint* arXiv:1804.02767, 2018.
- [10] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie. Feature pyramid networks for object detection. In *Proc. CVPR*, 2017.
- [11] A. Bochkovskiy, C.-Y. Wang and H.-Y. M. Liao. YOLOv4: Optimal speed and accuracy of object detection. *arXiv preprint* arXiv:2004.10934, 2020.

- [12] G. Jocher. Ultralytics YOLOv5. Software, 2020. <https://github.com/ultralytics/yolov5>.
- [13] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia. Path aggregation network for instance segmentation. In *Proc. CVPR*, 2018.
- [14] G. Jocher, A. Chaurasia and J. Qiu. Ultralytics YOLOv8. Software, 2023. <https://github.com/ultralytics/ultralytics>.
- [15] C. Feng, Y. Zhong, Y. Gao, M. R. Scott and W. Huang. TOOD: Task-aligned one-stage object detection. In *ICCV*, 2021. arXiv:2108.07755.
- [16] X. Li, W. Wang, L. Wu, S. Chen, X. Hu, J. Li, J. Tang and J. Yang. Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection. In *NeurIPS*, 2020. arXiv:2006.04388.
- [17] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding. YOLOv10: Real-time end-to-end object detection. In *Advances in NeurIPS*, 2024. arXiv:2405.14458.
- [18] L. Zhu, B. Liao, Q. Zhang, X. Wang, W. Liu, and X. Wang. Vision Mamba: Efficient visual representation learning with bidirectional state space model. In *Proc. ICML*, 2024. arXiv:2401.09417.
- [19] A. Hatamizadeh and J. Kautz. MambaVision: A hybrid Mamba–Transformer vision backbone. In *Proc. CVPR*, 2025. arXiv:2407.08083.
- [20] J. Liu, X. Fan, Z. Huang, G. Wu, R. Liu, W. Zhong, and Z. Luo. Target-aware dual adversarial learning and a multi-scenario multi-modality benchmark to fuse infrared and visible for object detection. In *Proc. CVPR*, 2022.
- [21] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang. ByteTrack: Multi-object tracking by associating every detection box. In *Proc. ECCV*, 2022.